

Bumi 综合使用文档（对外）

一、关于Bumi

（一）部件信息



Bumi 整机具备个21自由度。单臂拥有 4 个自由度，包括（左手坐标系）：

- 1. 肩关节pitch
- 2. 肩关节roll
- 3. 上臂关节yaw
- 4. 手肘关节pitch

单腿拥有 6 个自由度，包括：

- 5. 胯关节pitch
- 6. 髋关节roll
- 7. 大腿关节yaw
- 8. 膝关节pitch
- 9. 踝关节pitch
- 10. 踝关节roll

腰关节拥有一个自由度，包括：

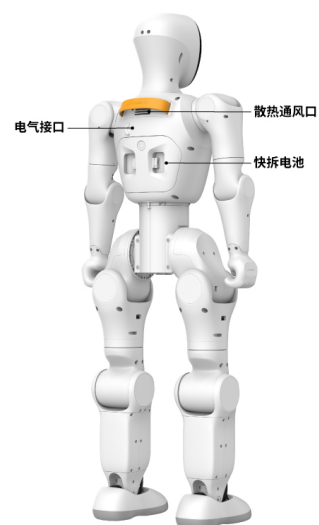
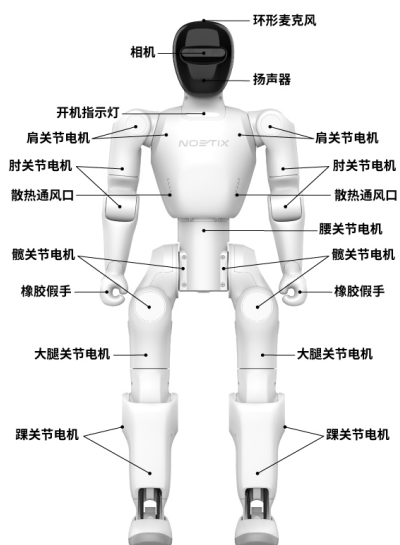
- 11. 腰关节yaw

拥有多个关节电机自由度，使得Bumi机器人能够实现精确的运动控制和姿态控制。

详细信息如下：

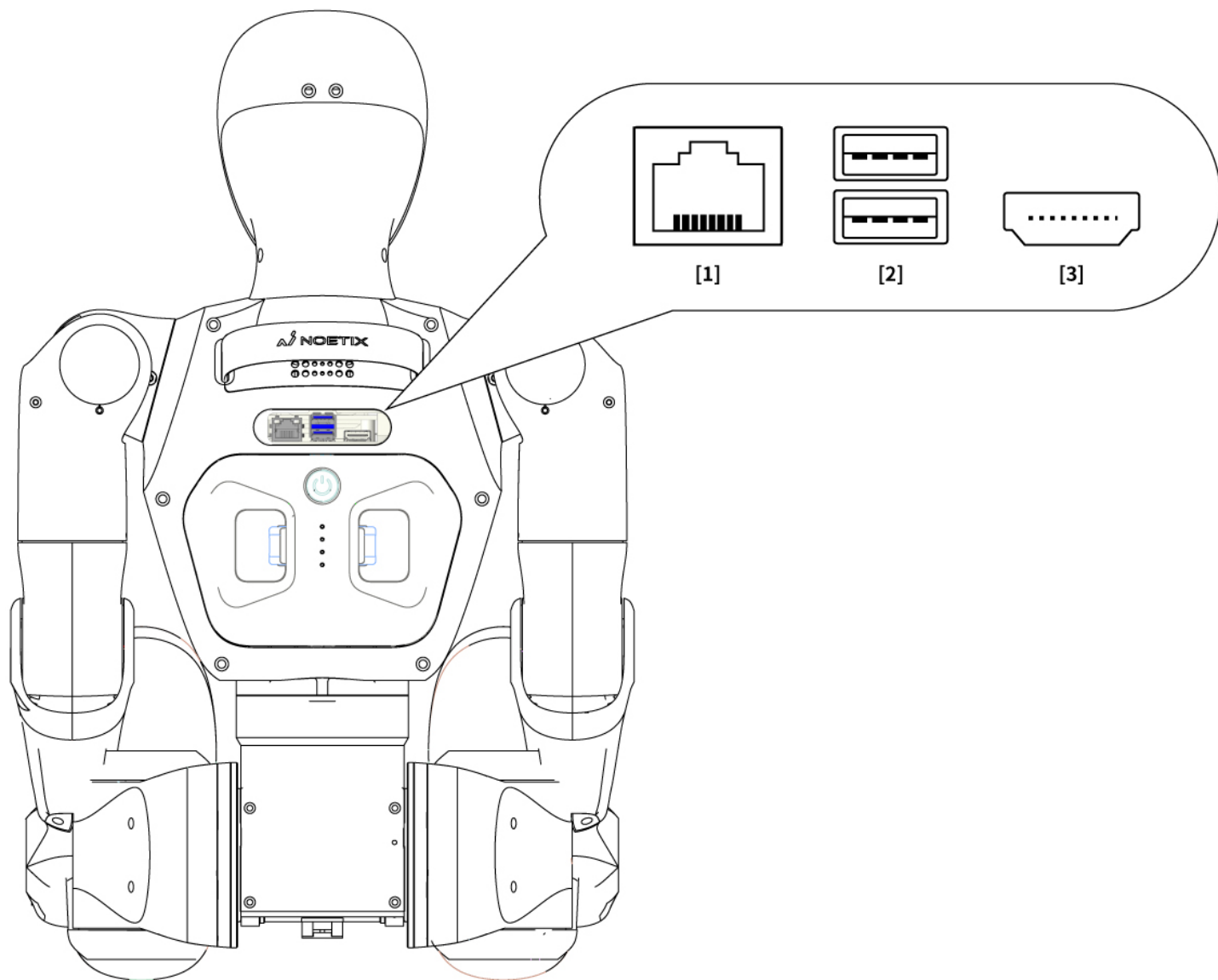
参数	Bumi	Bumi-教育版
高宽厚(站立)	98cm*35cm*20cm	
带电池重量	约17kg	
总自由度	21	

单腿自由度	6	
单手臂自由度	4	
腰部自由度	1	
算力模块	/	NVIDIA Jetson Orin （Nano Super / NX）
关节通讯方式	Ethercat & Canfd	
关节最大扭矩	70N.m	
小腿&大腿长度	约513mm	
手臂臂展	约379mm	
供电方式	锂电池	
基础算力	6 Tops	6 + 67 / 117 Tops
相机型号	DC_V93	Realsense D435ICB
感知传感器	IMU、相机	
麦克风阵列	标配	
扬声器	标配	
智能电池	快拆电池	
图像传输	App内有	二次开发中可以获取视频流
充电器	有	
手机APP	有	
OTA升级	有	有
续航时间	2~3H	
二次开发（highcontroller）	有	有
二次开发（lowcontroller）	/	有
部件介绍图		



(二) 电气接口

在Bumi背面，提供电气接口如下：



序号	类型	说明	数量
1	RJ45	交换机 1000 BASE-T	1
2	USB	运控板/算力板 USB 3.0	2
3	HDMI	运控板/算力板 高清显示线	1



TIP

带算力板的Bumi 后背接口为算力板的接口，不带算力板的Bumi，后背接口为运控板的接口

（三）机载计算机

类型	型号	用户	密码	IP	算力	频率	内存
----	----	----	----	----	----	----	----

运控板	瑞星微 RK3576	运控板不对用户开放		192.168.55.102	6 Tops	2.4GHz	8+64GB
二开算力板	NVIDIA Jetson orin nano super	noetix	bumi	192.168.55.101	67 Tops	1.7GHz	8+128GB
二开算力板	NVIDIA Jetson orin NX	noetix	bumi	192.168.55.101	117 Tops	2.0GHz	8+128GB

!

[CAUTION]

运控板（瑞星微 RK3576）不面向用户开放，如果用户私自访问运控板，导致机器人运控程序失控，造成损失与我司无关

(四) 相机信息

Bumi-EDU：Realsense D435i



<https://www.intel.cn/content/www/cn/zh/products/sku/190004/intel-realsense...>

英特尔® 实感™ 深度摄像头 D435i - 产品规格 | 英特尔

英特尔® 实感™ 深度摄像头 D435i 规格快速参考、功能和技术。

Bumi-标准：DC_V93

产品型号	USB 摄像模组	
感光片	(1/4")	
最高有效像素	2592(H) *1944 (V)	
Lens Size	1/4 inch (16 : 9)	
Pixel Size	1.4um *1.4um	
Image area	TBD	
输出图像格式	MJPEG / YUV2 (YUYV)	
支持的分辨率和帧率	2592*1944 @ 30fps MJPEG 1920x1080 @ 30fps MJPEG 1280x720 @ 30fps MJPEG 848x480 @ 30fps MJPEG 640x480 @ 30fps MJPEG	2592*1944 @ 2fps YUV 1920x1080 @ 5fps YUV 1280x720 @ 8fps YUV 848x480 @ 20fps YUV 640x480 @ 30fps YUV
相机板装配技术	SMT	
对焦	固定 /AF 对焦	
中心清晰度	800LW/PH (Center)	
信噪比	36dB	
动态范围	71.6dB	
灵敏度	1.4V/lux-sec@550nm	
最低照度	0.5lux	
快门类型	TBD	
接口类型	USB2.0 High Speed	
自动曝光控制 AEC	支持	
自动白平衡 AEB	支持	
自动增益控制 AGC	支持	
可调节参数	亮度 对比度 色饱和度 色调 清晰度 伽玛 白平衡 逆光对比 曝光度	
镜头规格	尺寸 1/4 焦距 6.78m 光圈 F2.4 32.6 ° (对角)	
灯板供电接口	/	

(五) 电池信息

参数	信息
品牌	深圳格瑞普

型号	GRP6752097-13S1P
容量	5100mAh
电压	48V

二、使用说明

(一) 启动设备及注意事项



一、设备取出与放置

1. 将 Bumi 从收纳位置取出。
2. 将其扶稳，使其坐于地面。

二、电池安装

3. 将电池插入 Bumi 后背电池仓。
4. 确认电池安装到位（卡扣锁定）。

三、设备开机

5. 短按一次电池开关，确认电池指示灯亮起。
6. 随后长按电池开关，直至出现流水灯效果。
7. 等待机器人完成开机。

四、初始化姿态调整

8. 将 Bumi 面朝上平躺于地面。

五、系统初始化说明

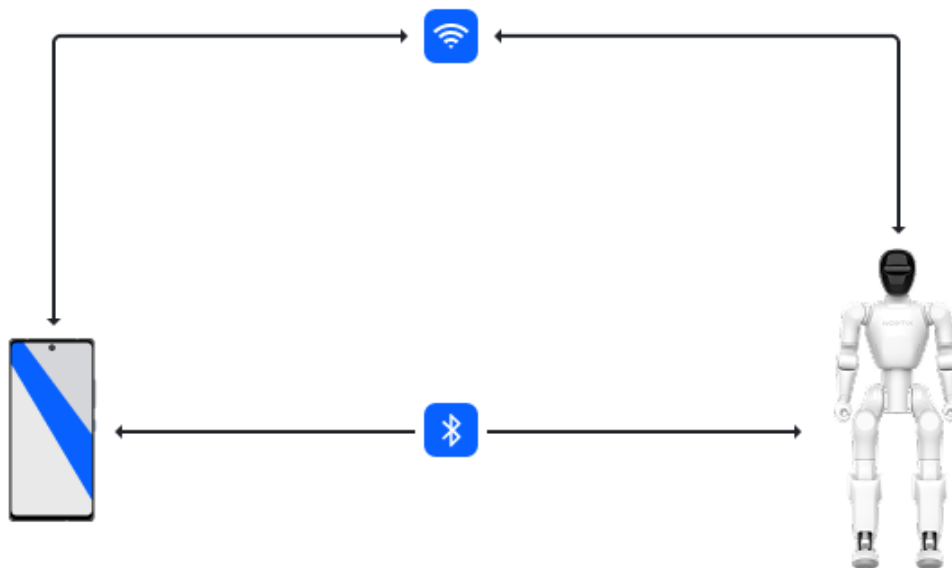
9. 开机后，Bumi 会自动进入全身使能模式：
 - 所有关节将自动运动至零位
 - 过程中请勿干预或阻挡关节运动

⚠ 注意事项

- 开机过程中避免触碰关节，防止夹伤或干扰初始化

- 确保地面平整，避免姿态异常
- 电池未正确安装时禁止开机

(二) Bumi连接方式



! [CAUTION]

连接 Bumi 后，请勿在系统设置中选择‘遗忘设备’，否则会清除蓝牙记录，导致该设备无法再次连接。如误操作，请联系售后。

(三) 手机APP连接设备及操作控制

Noetix run App下载：

Android:



iOS: [NOETIX Run App - App Store](#)

Noetix run App使用说明: [BUMI机器人使用说明](#)

(四) 设备关机及电池充电



一、设备收纳准备

1. 在 APP 中执行 **一键收纳**操作。
2. 收纳完成后，将 Bumi 扶稳，使其**坐于地面**。

二、设备关机

3. 长按电池开关，直至 Bumi 完全关机。

三、电池拆卸

4. 按住电池卡扣。
5. 从 Bumi 后背电池仓中**取出电池**。

四、电池充电

6. 将电池放置于电池底座上，并确保连接到位。
7. 将充电适配器接入电池底座。
8. 将适配器插入 **220V 电源**。

五、充电状态确认

9. 观察电池指示灯：

- 指示灯闪烁：正在充电
- 无指示：电池已充满

（五）操作教程视频

1. 安装APP



2. 开机

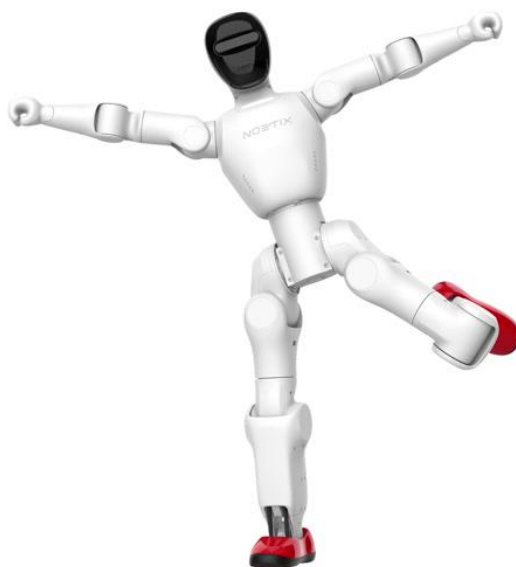
Bumi小布米 使用说明

开机



3. 激活绑定

Bumi小布米 使用说明



激活绑定

4. 连接BUMI

1. 蓝牙连接



Bumi小布米 使用说明

使用机器人(1)-蓝牙配对



2. 蓝牙组网和WIFI连接



Bumi小布米 使用说明

使用机器人(2)-蓝牙组网与WIFI连接



连接 Bumi 后，请勿在系统设置中选择‘遗忘设备’，否则会清除蓝牙记录，导致该设备无法再次连接。如误操作，请联系售后。

5. 一键起身



Bumi小布米 使用说明

一键启动机器人



! 使用Bumi的一键起身功能时 要确地面平坦不打滑，且周围无杂物，避免Bumi执行自动起身的时候被周围杂物影响而失败

6. 移动

Bumi小布米 使用说明

机器人移动



7. 基本动作

Bumi小布米 使用说明

基本功能



8. 示教模式

Bumi小布米 使用说明

示教



9. 舞蹈模式

Bumi小布米 使用说明

舞蹈



10. 图传模式



Bumi小布米 使用说明

图传

注意:开启图传后语音交互功能禁用, 关闭图传10s后语音交互启用

11. 语音交互



Bumi小布米 使用说明

智能识别交互

注意: 对准小布米头顶上方说话效果更好

12. 一键收纳

Bumi小布米 使用说明

一键收纳功能



！ 在APP中执行 **一键收纳** 前 要确保地面平坦不打滑 且周围无杂物

13. 关机

Bumi小布米 使用说明

关机



14. 电池充电



Bumi小布米 使用说明

充电

（四）常见问题及设备日常维护

1. 设备防水系数不高

请勿在雨雪天气室外或者潮湿的环境中使用

三、Bumi开发指南

✓ SDK介绍

本文档介绍Bumi基于 **DDS消息中间件** 通信开发，本SDK采用 **C/S架构**，在隔离机制下，使Bumi 机器人的 **二次开发算力板部署DDS客户端程序** 与机器人上作为服务器的 **运控板的运控程序** 以 **500HZ** 的频率下发控制命令，并持续获取当前机器人状态。dds采用的是 **CYCLONEDDS 0.11.0**。



注意：仅配有算力板版本的Bumi可以使用此SDK开发

(一) 系统架构图



(二) 开发接口

DDS C++

(三) 获取SDK

[GitHub - Noetix-Robotics/noetix_sdk_bumi](#): This is sdk for noetix robot Bumi with DDS interface. The

(四) URDF文件



meshes.rar
8.41MB



(五) 应用开发 Highcontrol



应用开发介绍

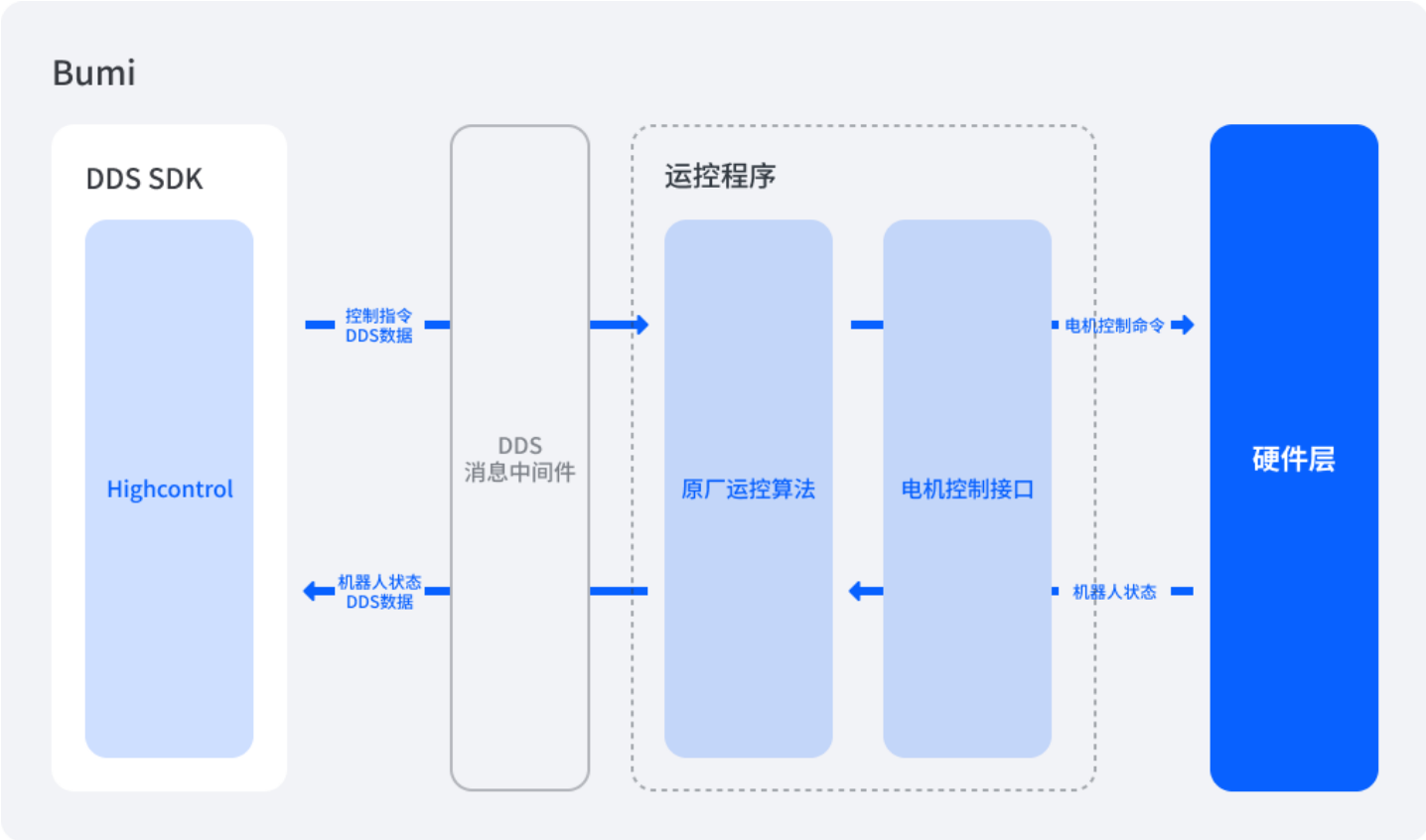
Highcontrol 基于原厂运动控制策略进行上层调度开发，在不改变底层控制逻辑的前提下，实现机器人自动化应用的快速构建，并支持与大模型集成，打造定制化智能行为系统。

典型应用包括：

- 结合视觉算法与原厂运控系统，实现自主感知与自主行进
- 结合语音交互模块，实现自然语言控制
- 接入大模型，根据输入动态生成并调用原厂控制动作（具备一定风险，需要严格安全策略）
- 配合外挂感知雷达，实现定位导航巡检系统

简而言之，HighControl 通过对原厂控制能力的高层调度封装，实现对遥控器或 APP 可控行为的自动化与程序化调用，并在此基础上拓展部分自主化与智能化能力。

1. Highcontrol架构图



(1) 状态上报机制

当客户端 SDK 与机器人运控程序建立连接后，运控程序会持续向客户端推送机器人实时状态信息，包括：

- 全身电机参数数据（**pos 位置、vel 速度、tau 扭矩、kp、kd、temperature 温度、error 错误信息**）
- IMU 状态数据
- 当前工作模式（**workmode**，表示当前运行的控制策略）
- 澳加狮手柄数据（用于同步人工输入或辅助控制）
- 电池电量及相关系统状态信息

(2) 客户端控制指令下发

客户端结合自身应用场景向运控程序发送控制指令，主要包括：

- (x, y, z) 行进控制数据**
仅在特定模式下有效，例如走路模式、握手、打招呼、欢呼、示教模式等。
- action 指令**
与 **workmode** 对应，用于切换或触发机器人当前运行的算法策略。

- **data 附加 DDS 数据**

通常在 **PLAYTEACH**（播放示教模式）下使用，用于指定示教文件索引等附加参数。

(3) 运控程序内部处理流程

运控程序接收到客户端指令后：

- 将控制参数传递给内部控制算法
 - 算法基于强化学习模型进行策略计算
 - 生成电机目标期望值（目标位置、速度或力矩）并发送到电机控制接口
 - 将计算结果下发至硬件控制层
-

(4) 硬件执行层

硬件层接收目标控制数据后：

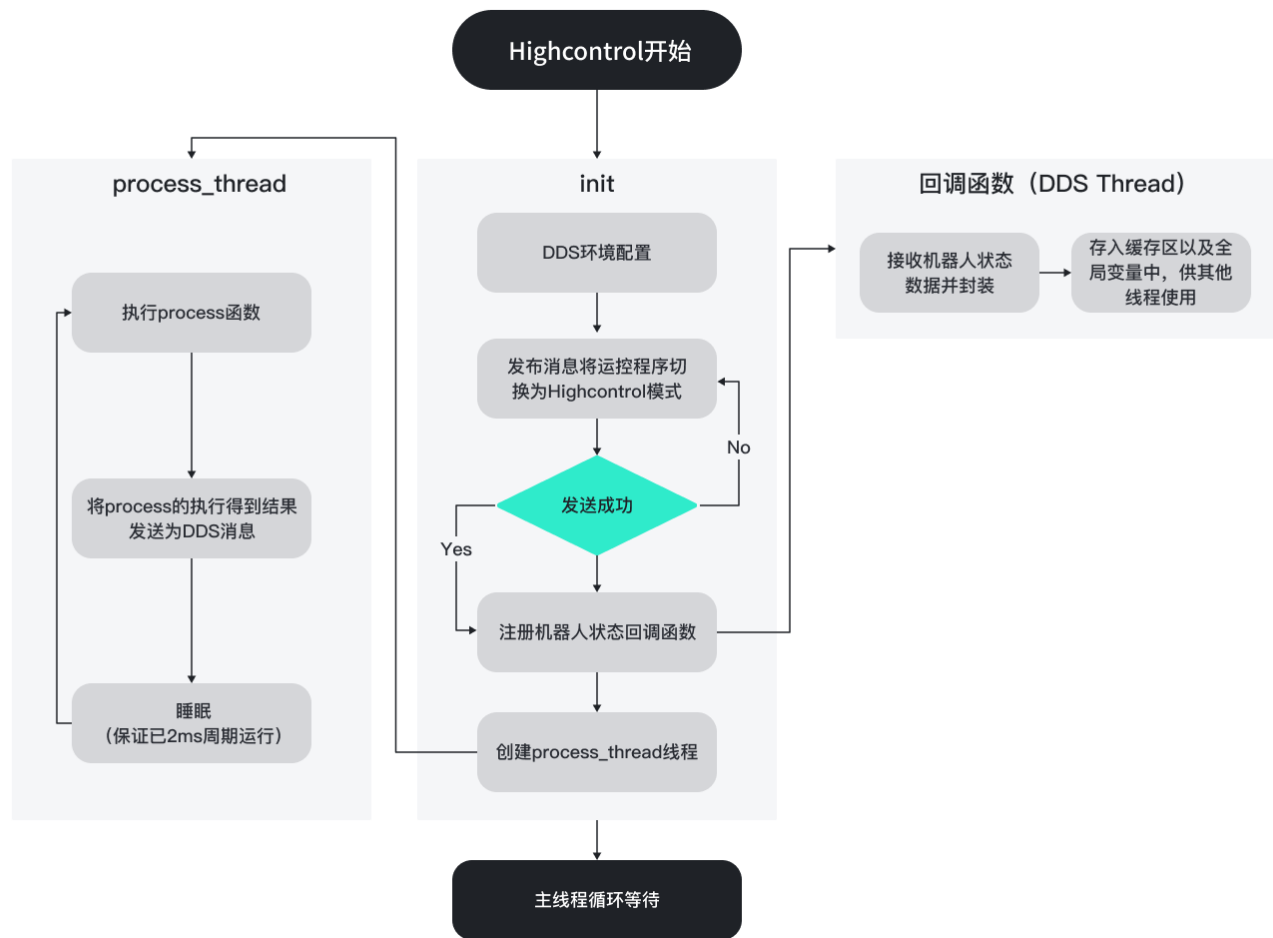
- 执行电机控制指令
 - 完成对应的期望动作
-

在整个流程中，用户仅需下发部分高层控制参数。

具体的运动规划、动态平衡与电机级控制计算均由原厂运控程序中的黑盒算法完成。

因此，在 **Highcontrol 模式** 下，用户能够调用和自动化的能力范围，原则上不超过 APP 或手柄原本可实现的功能边界，但不开放底层电机级控制权限。

2. Highcontrol运行流程图



(1) 初始化阶段 (init)

系统启动后完成以下初始化流程：

1. 建立客户端与机器人运控程序之间的通信连接
2. 注册机器人状态数据接收回调函数
3. 创建一个核心线程：
 - `process_thread`：控制计算线程

(2) 状态数据回调机制

运控程序持续向客户端推送机器人状态数据。

回调函数接收到数据后进行封装，并分别存入缓冲区及全局状态变量：

- `motor_state_buffer_`：电机状态缓冲区
- `imu_buffer_`：IMU 状态缓冲区
- `joy_buffer_`：手柄数据缓冲区
- `curmode`：当前运控策略模式（全局变量）

- `curbattery`：当前电池电量信息

该阶段仅负责数据接收与缓存，不进行控制决策计算。



TIP

带算力板的Bumi 后背接口为算力板接口，运控板无法插入手柄接收器，所以DDS获取不到手柄数据，所以手柄数据需要直接算力板上获取，详细请看 三 - (六) - 6

(3) `process_thread`（控制计算线程）

控制线程以 **2ms 固定周期**（包含 `process` 执行时间）循环运行：

1. 从各状态缓冲区读取最新数据
2. 执行 `process()` 控制逻辑
3. 生成控制命令数据
4. 将结果发送为DDS消息

该线程负责高层行为调度与控制决策生成。

(4) 整体架构特征

Highcontrol 模式采用典型的：

状态接收 → 缓存 → 周期计算 → 缓存 → 周期发送

的双线程异步架构。

特点包括：

- 状态与控制解耦
- 计算与发送分离
- 周期驱动（2ms / 500Hz）
- 基于原厂运控策略进行高层调度

控制边界仍受限于原厂运控策略能力范围，但通过线程化与缓冲区机制，实现自动化与程序化控制能力。

3. 参数表及常用接口使用

`RobotControlCmd::ControlCmd` DDS 命令下发类

代码块


```

1 RobotControlCmd::ControlCmd controlcmd;
2 controlcmd.axes()[0] = [机器人转向值:float]
3 controlcmd.axes()[1] = [机器人前进后退值:float]
4 controlcmd.action() = (int)[机器人模式切换枚举:int];
5 controlcmd.data() = [示教相关模式下附加索引值:int];

```

action 参数表示切换机器人到对应模式

action 对应下述 **ControlCmd** 枚举量映射，使用枚举提高代码可阅性，但是通过 **ddswrapper.publishControlCmdData()** 接口发送数据时需要**强转数据类型**

参数名称	定义原型	序号	示例值	说明
ControlCmd	代码块 <pre> 1 enum class ControlCmd : uint8_t { 2 WALK, 3 SWING, 4 SHAKE, 5 CHEER, 6 RUN, 7 START, 8 SWITCH, 9 STARTTEACH, 10 SAVETEACH, 11 ENDTEACH, 12 PLAYTEACH, 13 DANCE, 14 FALLTOSTAND, 15 STANDTOFALL, 16 DANCE1, 17 DANCE2, 18 TEAR, 19 DEFAULT 20 }; </pre>	0	WALK	走路模式
		1	SWING	出厂预设动作挥手
		2	SHAKE	出厂预设动作握手
		3	CHEER	出厂预设动作欢呼
		4	RUN	预留模式，当前版本不可用
		5	START	使能/失能
		6	SWITCH	准备模式
		7	STARTTEACH	开始示教
		8	SAVETEACH	保存示教
		9	ENDTEACH	结束示教（弃用）
		10	PLAYTEACH	播放示教
		11	DANCE	舞蹈1模式
		12	FALLTOSTAND	倒地起身
		13	STANDTOFALL	起身倒地
		14	DANCE1	舞蹈2模式

		15	DANCE2	舞蹈3模式
		16	TEAR	预设擦眼泪动作
		17	DEFAULT	空模式

✓ DEFAULT空模式使用目的

整个机器人线程会循环执行 `process` 控制函数是周期循环执行的，如果在其中持续下发

`action = ControlCmd::PLAYTEACH;` 示教播放命令的话

机器人会在 示教播放动作未播放完的情况，导致动作被反复打断，表现为机器人“卡住、无反应”。

正确做法

`PLAYTEACH` 属于 一次性触发指令（事件），应 只发送一次；后续循环发送 `DEFAULT`（空指令），避免重复触发。

示例代码（修正后）

```
if (is_first) {
    action = ControlCmd::PLAYTEACH; // 只触发一次
    is_first = false;
} else {
    action = ControlCmd::DEFAULT;
}
```

效果

- 收到 `PLAYTEACH` → 触发并完整执行示教动作
- 后续 `DEFAULT` → 不再打断动作
- 机器人行为正常、可预期

动作类指令要“边沿触发”，不能在控制循环里持续下发。

✓ START 使能/失能

`workmode == 30`，发送START使机器人进入使能模式

`Workmode != 30`，发送START使机器人立刻进入失能模式

代码块

```
1  if (key_updown[Key1] == 1 && key_updown[Key8] == 1 &&
2      (key_inuse[Key8] == 0)) { // 解析澳加狮手柄数据
3      action = ControlCmd::PLAYTEACH;
4      key_inuse[Key8] = 1; // 防止重复遥控被数据采样
5      index = 1; // 指定播放示教文件索引
6      printf("[DEBUG]: PLAYTEACH \n");
7  }
```

机器人控制命令下发示例

代码块

```
1  if (...) { // 控制逻辑
2      cmd.x = ...
3      cmd.yaw = ...
4      action = ...
5      index = ...
6  } else {
7      action = DEFAULT
8      cmd.x = 0;
9      cmd.yaw = 0;
10 }
11 RobotControlCmd::ControlCmd controlcmd;
12 controlcmd.axes()[0] = cmd.yaw;
13 controlcmd.axes()[1] = cmd.x;
14 controlcmd.action() = (int)action;
15 controlcmd.data() = fileindex;
16 ddswrapper.publishControlCmdData(controlcmd);
```



仅当 **action** 为以下模式时：

TEAR / PLAYTEACH / WALK / SWING / CHEER / SHAKE / DEFAULT

参数 **x** 和 **yaw** 才会生效，用于控制机器人运动；在其他模式下，**x** 和 **yaw** 无效。

在上述模式下，持续发送 **x** 和 **yaw**，机器人将按对应参数**持续移动!!!**

当再次发送 **x = 0** 且 **yaw = 0** 时，机器人停止移动。

使用注意：

- 停止机器人移动时，务必发送 **x = 0**、**yaw = 0**

接收数据 **workmode** 对应表

workmode值	说明
0	使能模式
1	准备模式
2	走路模式
5	舞蹈模式
8	出厂预设动作 打招呼
9	出厂预设动作 握手
10	出厂预设动作 欢呼
11	开始示教
12	结束示教
13	播放示教
14	保存示教
23	播放示教
26	保护模式
27	倒地起身
28	起身倒地
29	保存示教动作
30	失能模式
31	舞蹈模式1
32	舞蹈模式2
33	擦眼泪动作



保护模式

在处于**站立控制策略（走路，示教，舞蹈等）**下，运控程序会对 IMU 姿态数据进行持续监测。

当检测到姿态偏离站立稳定区间，并超过预设阈值时，系统将立即触发保护机制，强制切换至安全模式。

在保护模式下：

- 全身关节进入失能状态（电机停止主动输出）
- 终止当前运动控制策略
- 阻断进一步动作指令执行

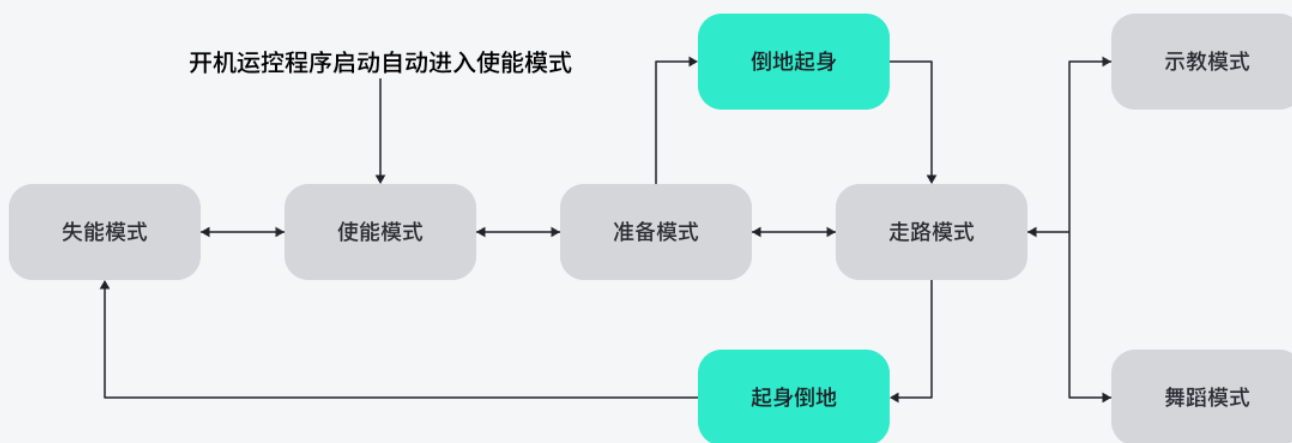
该机制的目的在于：

- 防止机器人在跌倒或失稳过程中持续输出控制力矩
- 避免四肢失控摆动造成二次损伤
- 降低对设备本体及周围环境的潜在损害

该保护逻辑属于底层安全防护机制，具有最高优先级，不受上层控制指令影响。

4. 运控状态切换流程图

BUMI 模式状态切换流程图



在除失能任何模式下，都可以通过急停将BUMI切换到失能模式

! 注意：使用highcontrol开发过程中，模式切换请遵循上述流程图以确保合理安全

5. 开发示例

以下例程功能，使用SDK Highcontroller中的示例demo，通过解析运控程序发送上来的澳加狮手柄按键数据，来切换机器人不同的模式，以及控制其行进方向。

代码块

```
1 void HighController::process() {
2     static int keyflag[14];
3     joydata remote_data;
4     static char key_updown[14], key_inuse[14];
5     Command cmd;
6     auto now = Clock::now();
7     long starttimestamp =
8
9     std::chrono::duration_cast<std::chrono::milliseconds>(now.time_since_epoch()).count();
10
11     const std::shared_ptr<const joydata> mc = joy_buffer_.GetData();
12     if (mc) {
13         for (int i = 0; i < 14; i++) {
14             remote_data.button[i] = mc->button[i];
15         }
16         remote_data.axes[0] = mc->axes[0];
17         remote_data.axes[1] = mc->axes[1];
18     }
19
20     for (int i = 0; i < 14; i++) {
21         if (remote_data.button[i] == 0) {
22             key_updown[i] = 0;
23             key_inuse[i] = 0; // 防止重复采样
24         } else if (remote_data.button[i] == 1) {
25             key_updown[i] = 1;
26         }
27     }
28
29     ControlCmd action = ControlCmd::DEFAULT;
30     cmd.x = remote_data.axes[1];
31     cmd.yaw = remote_data.axes[0];
32     if ((key_updown[Key9] == 1) && (key_inuse[Key9] == 0)) {
33         key_inuse[Key9] = 1; // 防止重复采样
34         action = ControlCmd::START;
35         printf("[DEBUG]: START \r\n");
36     } else if (key_updown[Key10] == 1 && (key_inuse[Key10] == 0)) {
37         action = ControlCmd::SWITCH;
38         key_inuse[Key10] = 1; // 防止重复采样
39         printf("[DEBUG]: SWITCH \n");
40     } else if (key_updown[Key2] == 1 && key_updown[Key5] == 1 &&
41                (key_inuse[Key5] == 0)) {
```

```
39         action = ControlCmd::WALK;
40         key_inuse[Key5] = 1; // 防止重复采样
41         printf("[DEBUG]: WALK \n");
42     } // 下面省略部分
43     RobotControlCmd::ControlCmd controlcmd;
44     controlcmd.axes()[0] = cmd.yaw;
45     controlcmd.axes()[1] = cmd.x;
46     controlcmd.action() = (int)action;
47     controlcmd.data() = fileindex;
48     ddswrapper.publishControlCmdData(controlcmd);
49 }
```

(六) 运控开发 Lowcontrol

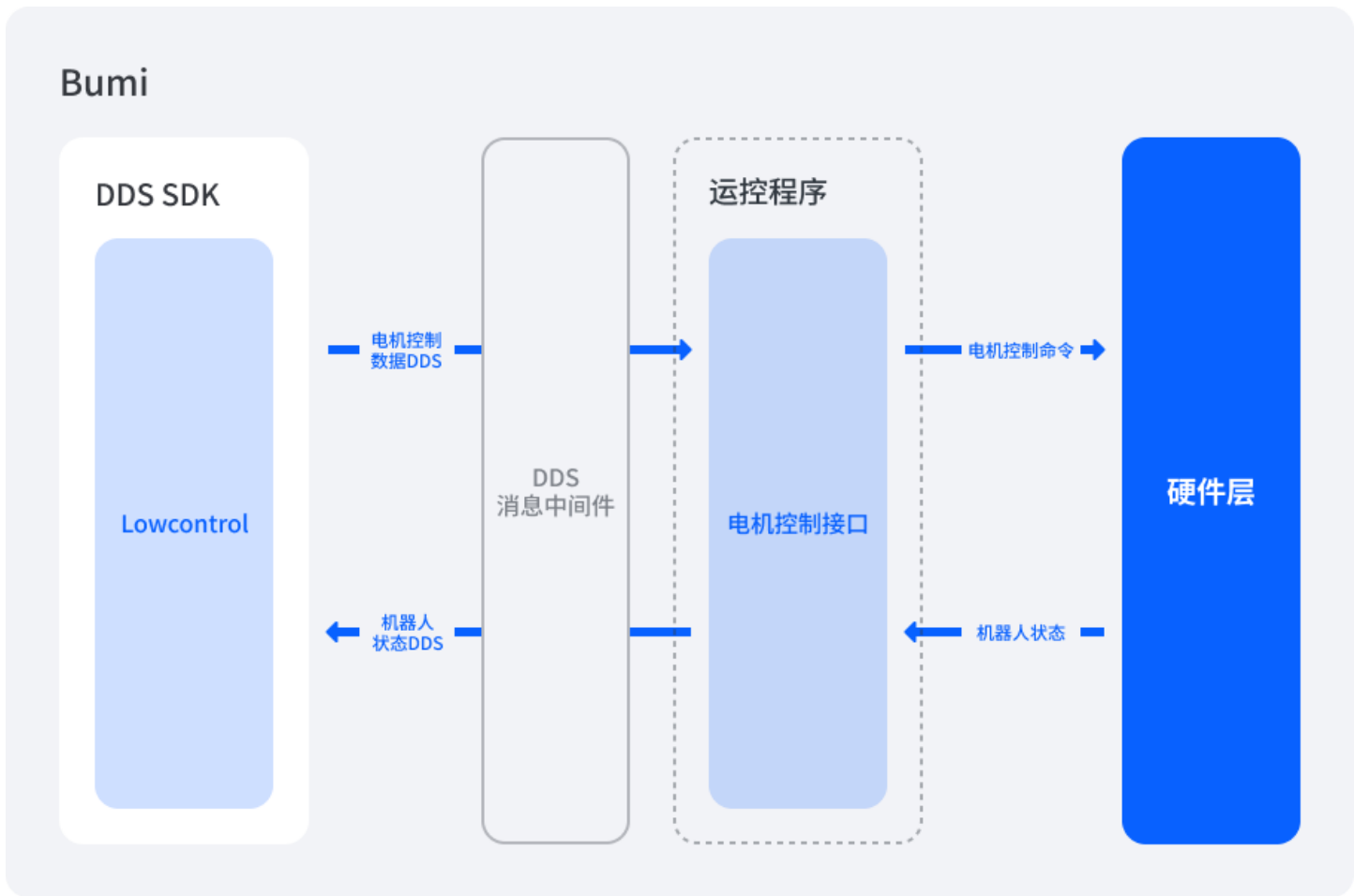
✓ 运控开发介绍

Lowcontrol 将运控程序中的电机控制接口开放至 SDK，允许用户以 500Hz 的控制频率直接调节底层电机参数。结合强化学习算法部署，可实现对 Bumi 运动能力的深度定制，使其完成多样化的新动作与舞蹈等复杂行为。

1. Bumi训练框架

后续开放

2. Lowcontrol架构图



(1) 状态数据上报

当客户端 SDK 与机器人运控程序建立连接后，运控程序会持续向客户端推送机器人实时状态信息，包括：

- 全身电机参数数据（**pos 位置**、**vel 速度**、**tau 扭矩**、**kp**、**kd**、**temperature 温度**、**error 错误信息**）
- IMU 状态数据
- 澳加狮手柄数据（用于同步人工输入或辅助控制）

(2) 客户端侧控制计算

客户端结合自身算法（通常为强化学习模型）进行实时策略计算，生成各电机的目标期望值（如目标位置等），并将电机控制指令数组写入缓冲区后下发至运控程序。

(3) 运控程序转发机制

运控程序接收到客户端下发的电机指令数组后，不进行高层策略计算，而是直接将控制参数转发至硬件控制层。

(4) 硬件执行层

硬件层接收电机目标参数后执行控制指令，驱动电机完成期望动作。

在 LowControl 模式下：

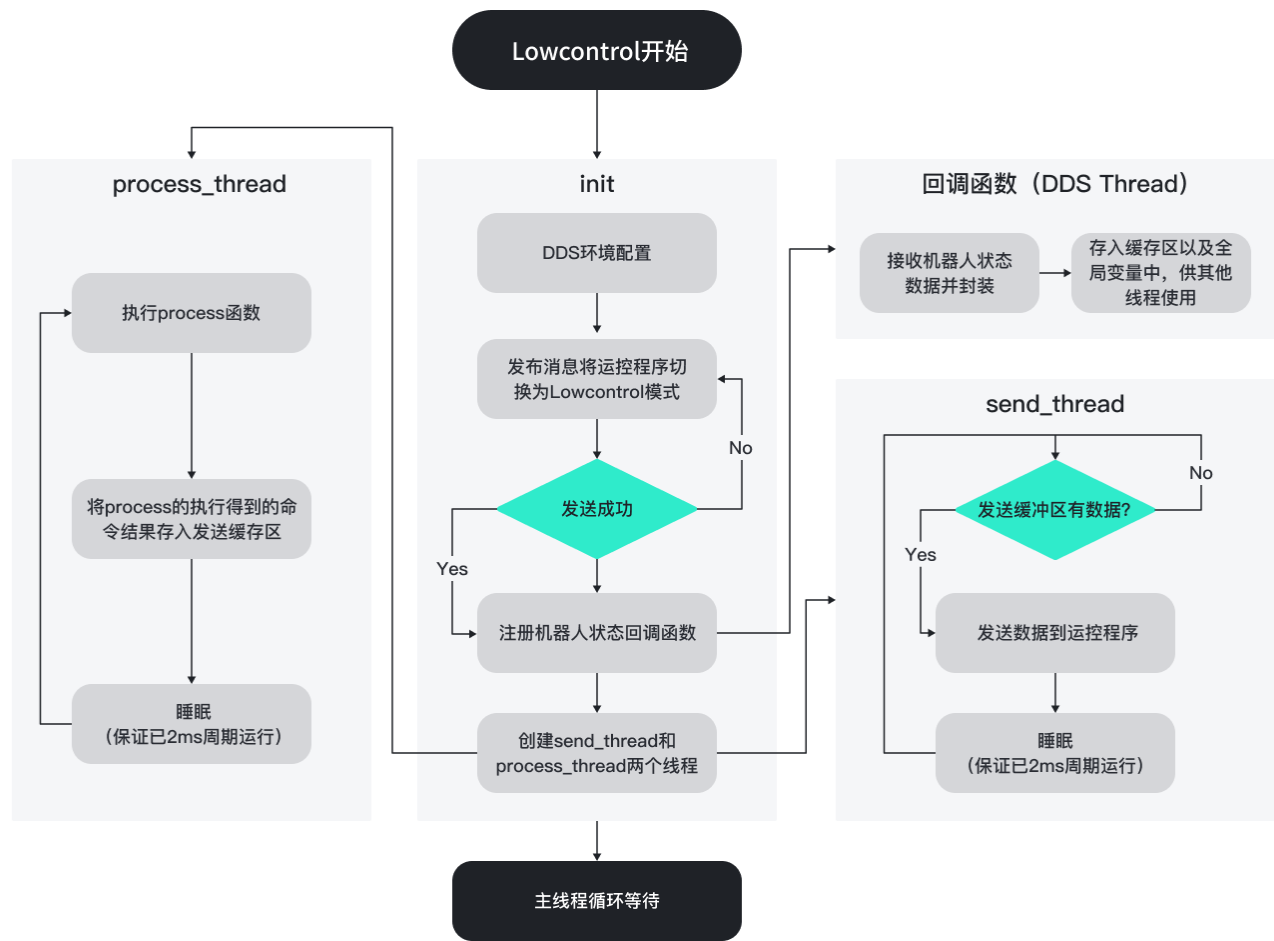
- 强化学习模型部署在客户端侧
- 用户可直接参与电机级控制
- 运控程序主要承担通信与安全转发职责
- 控制频率可达 500Hz

该模式具备更高的控制自由度与灵活性，但同时也意味着：

- 对算法稳定性要求更高
- 对实时性与系统安全性的要求更严格
- 若控制策略不稳定，可能直接影响机器人运动安全

3. Lowcontrol运行流程图

Lowcontrol流程图



(1) 初始化阶段 (init)

系统启动后完成以下初始化步骤：

- a. 建立客户端与机器人运控程序之间的通信连接
- b. 注册机器人状态数据接收回调函数
- c. 创建两个核心线程：
 - `process_thread`：控制计算线程
 - `send_thread`：数据发送线程

(2) 状态数据回调机制

运控程序持续向客户端推送机器人状态数据。

回调函数接收数据后进行封装，并存入对应缓冲区或全局变量：

- `motor_state_buffer_`：电机状态缓冲区
- `imu_buffer_`：IMU 状态缓冲区
- `joy_buffer_`：手柄数据缓冲区

该阶段仅负责数据接收与缓存，不参与控制策略计算。



TIP

带算力板的Bumi 后背接口为算力板接口，运控板无法插入手柄接收器，所以DDS获取不到手柄数据，所以手柄数据需要直接算力板上获取，详细请看 三 - (六) - 6

(3) `process_thread`（电机级控制线程）

控制线程以 **2ms 固定周期（包含 process 执行时间）** 循环执行：

- a. 从各状态缓冲区读取最新数据
- b. 执行 `process` 控制算法（通常为客户端部署的强化学习模型）
- c. 生成电机目标控制指令数组
- d. 将控制结果写入 `motor_cmd_buffer_` 缓冲区

在 LowControl 模式下，控制算法完全运行于客户端侧，直接生成电机级目标值。

(4) `send_thread`（数据发送线程）

数据发送线程同样以 **2ms 周期运行**：

- 若 `motor_cmd_buffer_` 不为空

- 读取最新电机控制指令
- 将电机目标数组发送至运控程序

运控程序仅负责数据转发至硬件层，不参与高层控制决策。

(5) 架构特征

Lowcontrol 模式的核心特征是：

- 客户端直接参与电机级控制
- 强化学习或自定义控制算法运行在客户端
- 运控程序不再负责高层策略计算，仅承担通信与安全转发职责
- 控制频率可达 500Hz

该模式提供更高的控制自由度与算法可塑性，但同时对：

- 实时性
- 算法稳定性
- 线程安全与调度精度

提出更高要求。

4. 关节电机信息

ID	名称	关节	扭矩	角度范围		弧度范围	
0	arm_l1_joint	左臂肩关节pitch	5 N.m	-135	135	-2.36	2.36
1	arm_l2_joint	左臂肩关节roll	5 N.m	-8	111	-0.14	1.94
2	arm_l3_joint	左臂关节yaw	5 N.m	-90	90	-1.57	1.57
3	arm_l4_joint	左臂肘关节pitch	5 N.m	-129	0	-2.26	0.00
4	leg_l1_joint	左腿髋关节pitch	60 N.m	-120	120	-2.09	2.09
5	leg_l2_joint	左腿髋关节roll	60 N.m	-38	90	-0.66	1.57
6	leg_l3_joint	左大腿关节yaw	15 N.m	-145	145	-2.53	2.53
7	leg_l4_joint	左腿膝关节pitch	60 N.m	0	128	0.00	2.24
8	leg_l5_joint	左腿踝关节pitch	30 N.m	-55	25	-0.96	0.44
9	leg_l6_joint	左腿踝关节roll	30 N.m	-10	10	-0.17	0.17

10	arm_r1_joint	右臂肩关节pitch	5 N.m	-135	135	-2.36	2.36
11	arm_r2_joint	右臂肩关节roll	5 N.m	-111	8	-1.94	0.14
12	arm_r3_joint	右臂关节yaw	5 N.m	-90	90	-1.57	1.57
13	arm_r4_joint	右臂肘关节pitch	5 N.m	-129	0	-2.26	0.00
14	leg_r1_joint	右腿髋关节pitch	60 N.m	-120	120	-2.09	2.09
15	leg_r2_joint	右腿髋关节roll	60 N.m	-90	38	-1.57	0.66
16	leg_r3_joint	右大腿关节yaw	15 N.m	-145	145	-2.53	2.53
17	leg_r4_joint	右腿膝关节pitch	60 N.m	0	128	0.00	2.24
18	leg_r5_joint	右腿踝关节pitch	30 N.m	-55	25	-0.96	0.44
19	leg_r6_joint	右腿踝关节roll	30 N.m	-10	10	-0.17	0.17
20	waist_1_joint	腰关节yaw	27 N.m	-90	90	-1.57	1.57

5. 澳加狮手柄数据映射

澳加狮手柄。button[]和按键映射表如下：

键位描述	Button index
右肩键	1
左肩键	2
X	5
Y	6
B	7
A	8
+	9
-	10
无	11
左摇杆按下	12

手柄摇杆映射	映射说明
axes[0]	右摇杆Y方向值
axes[1]	左摇杆X方向值

手柄摇杆数据范围

最小值	最大值
-1	1

6. 常用接口及使用案例

`DataBuffer` 类：

DataBuffer类	
SetData	设置缓冲区中数据
GetData	获取缓冲区中数据
Clear	清空缓冲区

代码块

```
1  template <typename T> class DataBuffer {
2      public:
3          void SetData(const T &newData) {
4              std::unique_lock<std::shared_mutex> lock(mutex);
5              data = std::make_shared<T>(newData);
6          }
7          std::shared_ptr<const T> GetData() {
8              std::shared_lock<std::shared_mutex> lock(mutex);
9              return data ? data : nullptr;
```

```
10         }
11         void Clear() {
12             std::unique_lock<std::shared_mutex> lock(mutex);
13             data = nullptr;
14         }
15     private:
16         std::shared_ptr<T> data;
17         std::shared_mutex mutex;
18     };
```

函数名	SetData
函数原型	void SetData(const T &newData)
功能概述	往缓冲区写入数据
参数	newData 数据（模板）
返回值	无
备注	

函数名	Clear
函数原型	void Clear()
功能概述	清除缓冲区里面的数据
参数	无
返回值	无
备注	

函数名	GetData
函数原型	std::shared_ptr<const T> GetData()
功能概述	从缓冲区获取数据
参数	无
返回值	数据（模板）
备注	

用于缓存各种数据，通过SetData 和 GetData()来获取数据地址

电机控制命令数组存入缓冲区

存入缓冲区后，会由 `send_thread` 线程自动发送

函数名	set_joint
函数原型	void set_joint(std::array<MotorCmd, 21> motorcmd)
功能概述	设置关节数据
参数	motorcmd 电机命令结构体
返回值	无
备注	

参数名称	定义原型	示例值	说明
MotorCmd	代码块<pre>1 struct MotorCmd { 2 double pos; 3 double vel; 4 double tau; 5 double kp; 6 double kd; 7 uint16_t motor_id; 8 };</pre>	pos	电机位置
		vel	电机转速
		tau	电机扭矩
		kp	pid参数
		kd	pid参数
		motor_id	电机ID

注意：Lowcontrol中一次将全部电机控制命令数据下发给运控程序

代码块

```
1  std::array<MotorCmd, 21> motorcmd;
2  // 省略算法逻辑 ...
3  set_joint(motorcmd);
```

获取当前机器人电机状态数据

函数名	get_joint_state
函数原型	const std::array<MotorState, 21> get_joint_state()
功能概述	获取关节数据
参数	无
返回值	const std::array<MotorState, 21> 电机状态结构体数组
备注	

代码块

```
1  std::array<MotorState, 21> joint_state = get_joint_state();
```

获取IMU数据

参数名称	定义原型	示例值	说明
NingImuData	代码块<pre>1 struct NingImuData { 2 double ori[4]; 3 double ori_cov[9]; 4 double angular_vel[3]; 5 double angular_vel_cov[9]; 6 double linear_acc[3]; 7 double linear_acc_cov[9]; 8 };</pre>	ori	
		ori_cov	
		angular_vel	
		angular_vel_cov	
		linear_acc	
		linear_acc_cov	

代码块

```
1  const std::shared_ptr<const NingImuData> idata = imu_buffer_.GetData();
```


获取澳加狮手柄数据（无算力板的Bumi DDS获取）

参数名称	定义原型	示例值	说明
joydata	代码块 <pre>1 struct joydata { 2 double axes[2]; 3 int button[14]; 4 };</pre>	axes	遥控器遥感线性数值
		button	遥控器按键按下映射数值

代码块

```
1 const std::shared_ptr<const joydata> mc = joy_buffer_.GetData();
```

获取澳加狮手柄数据（带算力板的Bumi 算力板直接获取）

AoLionDriver 类

AoLionDriver类接口	
init	初始化
getremotedata	获取手柄数据

函数名	init
函数原型	bool init(std::string port, int baudrate)
功能概述	初始化手柄设备端口
参数	- port：设备端口号，默认为/dev/input/js0 - baudrate：波特率，默认为115200
返回值	true：初始化成功 falese：初始化失败
备注	

函数名	getremotedata
函数原型	const joydata getremotedata()
功能概述	获取手柄数据
参数	无
返回值	joydata：手柄数据结构体
备注	

使用示例

代码块

```
1  AoLionDriver aoliondriver;
2  aoliondriver.init("/dev/input/js0", 115200);
3  const joydata l_jdata=aoliondriver.getremotedata();
4  const joydata * jdata = &l_jdata;
5  if (jdata) {
6      memcpy(remote_data.button, &(*jdata).button[0],
7              sizeof(remote_data.button));
8      memcpy(remote_data.axes, &(*jdata).axes[0],
9              sizeof(remote_data.axes));
10     cmd.x = remote_data.axes[1];
11     cmd.y = 0;
12     cmd.yaw = remote_data.axes[0];
13 }
```

7. Demo例程介绍

`lowcontroller.cpp` 中运行了一个Bumi的21个自由度跑步策略（`policy/policy.onnx` 此policy 不能后退，会失控）的sim2real的代码，根据策略提供了部分强化学习部署接口

函数名	loadModel
函数原型	bool loadModel(std::string modelpath)

功能概述	加载自有模型
参数	modelpath: 模型路径（程序所在的当前绝对路径）
返回值	true：加载成功；false：加载失败
备注	

函数名	onnxdatainit
函数原型	void onnxdatainit()
功能概述	初始化onnx数据
参数	无
返回值	无
备注	

函数名	getmodelparam
函数原型	bool getmodelparam()
功能概述	从yaml文件中获取模型参数
参数	无
返回值	true：读取yaml文件成功；false：读取yaml文件失败
备注	

函数名	setparameter
函数原型	void setparameter(Command &cmd,bool* isfirst)
功能概述	设置xyz整机控制参数
参数	cmd：设置指令

	• isfirst： 是否是第一次设置
返回值	无
备注	

函数名	updatestateestimation
函数原型	bool updatestateestimation()
功能概述	更新状态，包括关节角度，IMU数据等从yaml文件中获取模型参数
参数	无
返回值	true：读取yaml文件成功；false：读取yaml文件失败
备注	

函数名	computeObservation
函数原型	void computeObservation()
功能概述	计算观测系数
参数	无
返回值	无
备注	

函数名	computeActions
函数原型	void computeActions()
功能概述	计算最终的动作
参数	无
返回值	无

备注	
----	--

手柄操作流程

使用步骤	
+	全身关节回0位进入阻尼模式
LB & -	四肢微微弯曲，进入策略前准备模式
LB & X	进入策略控制跑步模式

(七) 快速开发

以下所有操作都是在Bumi算力板上进行，使用DDS SDK开发前请先部署环境 或 在开发环境docker 中进行开发

部署环境请看：[📖 SDK 编译环境部署流程](#)

1. docker开发环境镜像部署



noetix_dds.tar.gz

524.99MB



 **注意：此镜像包为aarch64架构，仅在aarch64架构电脑上可部署**

1. 解压压缩包

代码块

```
1 tar -zxvf noetix_dds.tar.gz
```

解压出来后，压缩包内会有 `noetix_dds.tar` 开发镜像文件 `noetix_dds.yaml` 配置部署文件

```
1  version: "3.8"
2
3  services:
4    noetix_dds:
5      image: noetix_dds:arm64
6      container_name: noetix_dds
7      network_mode: host
8      volumes:
9        - /home/noetix/work:/home/noetix/work:rw # 挂载工作目录
10         # /home/noetix/work 冒号前这部分要改成用户的工作目录，创建一个/home/noetix/work/路径目录
11      stdin_open: true
12      tty: true
13
```

2. 部署

代码块

```
1  docker load -i noetix_dds.tar
2  docker images # 确认noetix_dds arm64 镜像是否存在
3  mkdir -p /home/noetix/work # 创建工作目录， 用户自行选择
4  docker compose -f noetix_dds.yaml up -d # 部署
5  docker ps # 检查 noetix_dds 是否存在
```

3. 进入dock容器

代码块

```
1  docker exec -it noetix_dds /bin/bash
```

后续将DDS SDK 放入工作目录即可开始开发

2. 编译SDK

代码块

```
1  cd noetix_Bumi_sdk
2  ./build_release.sh # 一键编译脚本
```

编译成功后会在SDK根目录下生成可执行文件 `highcontrol` 和 `lowcontrol`

3. 启动程序

代码块

```
1 cd noetix_Bumi_sdk
2 sudo ./lowcontrol # or sudo ./highcontrol
```



安全提示

调试过程中，请全程使用机器人吊架，以免错误程序失控造成损失

(八) 常见问题及注意事项

1. DDS客户端demo：运行报错(1)

ARM架构：

error while loading shared libraries: libssl.so.1.1: cannot open shared object file: No such file or directory



libssl1.1_1.1.1f-
1ubuntu2.16_arm64.deb
1.10MB



代码块

```
1 sudo dpkg -i libssl1.1_1.1.1f-1ubuntu2.16_arm64.deb
```

X86架构：

./highcontrol: error while loading shared libraries: libssl.so.1.1: cannot open shared object file: No such file or directory



libssl1.1_1.1.1f-
1ubuntu2_amd64.deb
1.26MB



代码块

```
1 sudo dpkg -i libssl1.1_1.1.1f-1ubuntu2_amd64.deb
```